

Continuous Integration Pipeline with Testing in Jenkins

lac – Jenkins- Assessment

Title: Continuous Integration Pipeline with Testing in Jenkins	2
Duration	2
Type.....	2
What you will be provided.....	2
What you need to know	3
Scenario	3
What you will do.....	3
Activities.....	3
Expected Outcome	4
Testcases	4

Title: Continuous Integration Pipeline with Testing in Jenkins

Duration

120 minutes

Type

Assessment

What you will be provided

- Pre-configured **VM** with:
 - Jenkins installed and running on port 8080
 - Java and Maven preinstalled
 - Required Jenkins plugins (Git, JUnit, Maven Integration, etc.)
- Sample Java Maven project in `/home/ubuntu/java-maven-app`

What you need to know

- Basic Git and GitHub operations (push, commit, clone)
- Maven build lifecycle (clean, package, test)
- Jenkins Freestyle job setup
- Configuring Poll SCM triggers
- Managing Jenkins build artifacts and reports

Scenario

Your task is to implement a **Continuous Integration (CI) pipeline** that automates build, test, and artifact management for a Maven-based Java project.

You are provided with a pre-configured **virtual machine** containing:

- **Java JDK** and **Maven** installed
- **Jenkins** with required plugins preinstalled

What you will do

- Push the provided project code to your **own GitHub repository**.
- Configure **Jenkins** to:
 - Clone the GitHub repository
 - Build the project using **Maven**
 - Run **unit tests** automatically
 - Publish **JUnit test reports**
 - Archive the generated **.jar artifact**

- Configure the Jenkins job to **automatically trigger builds using Poll SCM**.
- Validate that the CI pipeline runs automatically whenever new commits are pushed to the repository.

Activities

Step 1: Push project to GitHub

- Install Maven in the terminal.
- Initialize Git and push the project to a new GitHub repository in your own account.

Step 2: Create Jenkins job

- Browse <http://localhost:8080> for jenkins page.
- Login with details **Username: jenkinsuser , password: Jenkinsuser@123**
- In Jenkins, create a **Freestyle job** named **MavenTestJob**.
- Configure the job to pull source code from the **main** branch of your GitHub repository.

Step 3: Build and test

- Configure the job to execute Maven build and test phases.

Step 4: Reporting and Artifacts

- Configure Jenkins to:
 - Publish JUnit test reports from `target/surefire-reports/*.xml`
 - Archive the generated JAR artifact from `target/*.jar`

Step 5: Configure trigger (Poll SCM)

- Configure **Poll SCM** with the schedule: `H/5 * * * *`

Step 6: Verify automation

- Push a new commit and wait for Jenkins to automatically detect changes and start the build.
- Confirm the build executes `mvn clean package`, runs tests, and publishes reports.

Expected Outcome

- A Jenkins job named `MavenTestJob` is created.
- Maven build (`mvn clean package`) and test (`mvn test`) run successfully.
- Test reports are visible in Jenkins.
- Jenkins job automatically triggers within 5 minutes of a Git commit.
- The `.jar` artifact is archived as a build artifact.

- The latest build should be triggered and successful using poll SCM.

Testcases

1. Verifies whether the Jenkins job named MavenTestJob exists in Jenkins. (10 marks)
2. Validates that Maven build and test steps have executed successfully in Jenkins. (10 marks)
3. Checks if JUnit test reports have been published for the last completed build. (20 marks)
4. Ensures the Poll SCM trigger is correctly configured in Jenkins with schedule H/5 * * * *. (20 marks)
5. Validates that the .jar artifact from the Maven build is archived in Jenkins. (20 marks)
6. Confirms that the Jenkins job was triggered automatically by a Poll SCM event (not manually)(20 marks)